

“Design and Development of LUMINA – An Academic Research Repository for Scholarly Paper Submission, Peer Review and Discovery”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

NIMMA LOKESH REDDY : 2520030366

RAMAGIRI RISHIK RAO : 2520030333

Under the Esteemed Guidance of

Ch. Pavithra

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Design and Development of LUMINA – An Academic Research Repository for Scholarly Paper Submission, Peer Review and Discovery”** is a record of Bonafide work of **NIMMA LOKESH REDDY - 2520030366, RAMAGIRI RISHIK RAO - 2520030333** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

NIMMA LOKESH REDDY – 2520030366
RAMAGIRI RISHIK RAO – 2520030333

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project based report entitled **“Design and Development of LUMINA – An Academic Research Repository for Scholarly Paper Submission, Peer Review and Discovery”** is a bonafide work done and submitted by **NIMMA LOKESH REDDY - 2520030366, RAMAGIRI RISHIK RAO - 2520030333** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2025-2026**.

Signature of the Guide
(Ch. Pavithra)

Signature of the Course Coordinator Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish is to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our Course Coordinator and our guide **Ch. Pavithra**, Department of Computer Science Engineering, which cannot be expressed in words for her tremendous support, encouragement and guidance throughout this project.

We express our gratitude to **Dr. P Venkateswara Rao (CSE) / Dr. Kaja Shareef (CSIT)**, Head of the *Department for Computer Science Engineering / Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff, and also who have assisted us directly or indirectly for the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

NIMMA LOKESH REDDY – 2520030366
RAMAGIRI RISHIK RAO – 2520030333

Abstract

The volume of scholarly and academic research produced inside universities continues to grow every year, yet a large share of it remains scattered across personal drives, email threads, and informal group chats with no central, searchable, or quality-controlled home. Students and early researchers frequently struggle to discover prior work done within their own institution, to share their own papers in a credible way, and to control who is allowed to access sensitive or unpublished material. To address this gap, our project focuses on the Design and Development of LUMINA, an Academic Research Repository – a single, unified, web-based platform that streamlines the entire lifecycle of a research paper, from submission and review to discovery and controlled access.

LUMINA is built as a modern full-stack web application. The frontend is implemented with React 18 and Vite using React Router v6 for client-side navigation, while the backend is a RESTful API built with Node.js and Express.js, backed by a relational MySQL database. The platform is organized around several fully interconnected modules: a guided seven-step Upload Wizard that lets researchers submit papers along with optional source code; an automated metadata extraction engine that reads PDFs to pre-fill the title, authors, and abstract; a powerful browse-and-search experience with category, year, license, and visibility filters supported by a database full-text index; and a complete admin console for approving, rejecting, and auditing every submission.

Security and integrity are central to the design. Authentication uses JSON Web Tokens with bcrypt-hashed passwords, while administrator access is additionally protected by a two-factor one-time password delivered over Telegram. Every uploaded file is fingerprinted with a SHA-256 hash to detect duplicate or plagiarised submissions, and a role-based access-control model distinguishes guests, researchers, and administrators, with public, restricted, and private visibility levels enforced on every request. A gamified badge system rewards verified contributors, a weekly view-tracking mechanism powers a fair trending algorithm, and time-limited share links allow private papers to be shared safely.

In summary, LUMINA delivers a complete, reliable and user-friendly digital solution that significantly improves the way academic work is submitted, evaluated, preserved, and discovered within an institution. The system is responsive across devices, follows established web-security practices, and is structured so that new modules – such as citation graphs, recommendation engines, and external indexing – can be added with minimal effort.

INDEX

S. No.	Chapters	Topics
		Acknowledgement
		Abstract
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users / audience
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks
3	Technology Stack	3.1 Front-end (React.js, Vite) 3.2 Back-end (Node.js, Express.js) 3.3 Database (MySQL) 3.4 Version Control (Git, GitHub) 3.5 APIs / External services (Telegram, SMTP, arXiv)
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture
5	Design	5.1 ER Diagram / Database schema
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations
12	Conclusion	12.1 Summary of the project 12.2 What was achieved

S. No.	Chapters	Topics
		12.3 Skills learned during development
13	References	- Books, tutorials, APIs, documentation sites used
14	Appendices	- Screenshots of the app - Sample code snippets - Installation / setup instructions - User manual or guide - Geo Tag photos with guide - Review forms with guide signatures

1. INTRODUCTION

1.1 Background of the Project

Academic institutions generate an enormous quantity of intellectual output every single year. Final-year projects, conference papers, journal articles, seminar reports, dissertations, and undergraduate research all add to a growing body of scholarly knowledge. Despite this, most of that work is never preserved in any structured, institution-owned system. It usually lives on a student's laptop, inside an email attachment to a guide, or on a shared drive that becomes inaccessible the moment that student graduates. The result is a silent and continuous loss of knowledge: the same problems are re-solved year after year, juniors cannot learn from the work of seniors, and the institution has no consolidated record of its own research footprint.

LUMINA was conceived to solve precisely this problem. The name reflects the project's goal of bringing academic work out of the dark and into a well-lit, searchable, and credible public space. LUMINA is a centralised Academic Research Repository: a web platform where any verified researcher can submit a paper, where every submission is reviewed by an administrator before becoming public, and where any reader can search, filter, and discover existing work. It behaves like a miniature, institution-focused version of large-scale repositories such as arXiv or an institutional digital library, but it is purpose-built for a college environment with controlled access, moderation, and contributor recognition.

The system is implemented as a modern single-page web application supported by a RESTful backend service. The choice of a web platform is deliberate: it requires no installation, works on any device with a browser, and can be hosted centrally so that every member of the institution accesses the same up-to-date data. The frontend is built using React, a component-based JavaScript library, while the backend uses Node.js and Express to expose a clean API surface over a MySQL relational database.

It is worth contrasting LUMINA with the informal arrangements it is intended to replace. In a typical department, a student who has completed a project hands a soft copy to their guide, perhaps uploads it to a shared cloud folder, and then moves on. The folder accumulates hundreds of loosely named files with no metadata, no search, and no notion of approval. When a junior later wants to study previous work on, say, a recommendation system, there is no reliable way to locate it. LUMINA replaces this with a structured database in which every paper carries a title, an abstract, a set of keywords, a category, an author, an approval status, and a content fingerprint – all of which make the work findable, trustworthy, and permanent.

Development Methodology

The project followed an incremental, iterative methodology. The work was divided into clear milestones – database design, authentication, the upload pipeline, the moderation console, search and discovery, and access control – and each milestone was built, tested, and integrated before moving on to the next. This kept the system in a working state at every stage and allowed problems to surface early. Version control with Git underpinned the entire process, letting the two team members work in parallel on separate feature branches and merge their contributions with a complete history of every change.

1.2 Problem Statement

The existing approach to handling academic research within most colleges is fragmented and informal. The specific problems that LUMINA addresses can be summarised as follows:

- No central repository: Research papers are scattered across personal storage, making institutional knowledge effectively invisible and impossible to search in one place.
- No quality control: When work is shared informally there is no review step, so there is no guarantee that what circulates is genuine, complete, or correctly attributed.
- Poor discoverability: Even when papers exist, there is no search facility, no categorisation, and no way to find related work by topic, year, or author.
- No access control: Sensitive or unpublished work cannot be shared selectively. It is either fully private (and therefore useless to others) or fully exposed.
- No contributor recognition: Students who consistently produce quality research receive no visible, verifiable credit for their contributions.
- Duplication and plagiarism risk: Without fingerprinting of files there is no way to detect when the same document is submitted twice, possibly by different people.

LUMINA is designed to directly eliminate each of these problems through a unified, moderated, access-controlled, and searchable platform.

1.3 Scope of the Project

The scope of LUMINA covers the full lifecycle of an academic paper inside an institution. The following capabilities fall within scope:

- User registration and authentication for researchers, with secure password storage and token-based sessions.
- A guided, multi-step Upload Wizard that accepts a PDF paper and an optional source-code archive, automatically extracts metadata, and detects duplicates.

- An administrator console with two-factor authentication for approving, rejecting, and deleting submissions, with every action recorded in an audit log.
- Browsing, searching, and filtering of approved papers by category, year, license, and visibility, supported by a full-text search index.
- Access control across public, restricted, and private visibility levels, including private share links and explicit access-request workflows.
- A contributor badge system, weekly view tracking, and a trending algorithm based on genuine reader interest.
- Email and Telegram notifications for important events such as approval, rejection, and admin login.

The following items are explicitly outside the current scope and are noted as future enhancements: automated plagiarism scoring against external databases, full citation-graph analysis, a recommendation engine driven by machine learning, and integration with external indexing services such as Google Scholar or ORCID synchronisation.

In terms of concrete deliverables, the project produces a working frontend application, a fully functioning REST API, a populated relational database with a documented schema, and the supporting documentation contained in this report. Together these constitute a complete, demonstrable system rather than a prototype: every feature described in the scope is implemented end-to-end and can be exercised live.

1.4 Objectives

The primary objectives that guided the design and development of LUMINA are:

1. To design and implement a single, unified web platform that consolidates all academic research of an institution in one searchable place.
2. To enforce a moderation workflow so that every published paper has been reviewed and approved by an authorised administrator.
3. To secure the system using industry-standard practices including hashed passwords, JWT-based sessions, parameterised database queries, and two-factor administrator authentication.
4. To guarantee data integrity by fingerprinting every uploaded file with a SHA-256 hash for duplicate detection.
5. To provide flexible, fine-grained access control with public, restricted, and private visibility and time-limited share links.
6. To encourage participation through a transparent contributor badge system and a fair, view-based trending mechanism.

7. To build the system in a modular, maintainable way so that future features can be added with minimal disruption.

1.5 Importance of the Application

The importance of LUMINA lies in the long-term value it creates for an institution. By preserving research in a structured repository, the institution builds a permanent, growing archive of its own intellectual output that survives the graduation of any individual student. Juniors gain the ability to learn directly from the documented work of seniors, which raises the overall quality of new projects and reduces wasted effort on already-solved problems.

The moderation workflow ensures that the repository maintains credibility: because every paper passes through an administrator, the published collection can be trusted as genuine institutional work. The contributor badge system creates a positive incentive loop, motivating students to produce and submit quality work in exchange for visible recognition. Finally, the strong security and access-control model means the platform can safely hold even sensitive or unpublished material, which dramatically widens the range of work that can be stored.

The contrast between the existing informal arrangement and the proposed system makes the value concrete:

Existing informal approach	LUMINA (proposed system)
Papers scattered across drives and email	All papers in one searchable repository
No review – anything circulates	Every paper approved by an administrator
No search or categorisation	Full-text search with category/year filters
All-or-nothing sharing	Public, restricted, and private visibility
No record of contribution	Badges and verifiable contributor history
Duplicates go unnoticed	SHA-256 fingerprinting detects duplicates
Work lost when a student leaves	Permanent institutional archive

1.6 Target Users / Audience

LUMINA is designed for three clearly distinct categories of user, each with a different set of permissions and a different relationship to the system:

Guests (unauthenticated visitors): Any person who visits the platform without logging in. They can browse and search the catalogue of public papers and read their metadata, but cannot access restricted or private content, and cannot upload.

Researchers (registered users): Students, scholars, and faculty who register on the platform. They can upload papers through the Upload Wizard, manage their own submissions, request access

to private papers belonging to others, earn badges, and view their personal dashboard with notifications.

Administrators: Trusted moderators who log in through a separate two-factor flow. They review the queue of pending submissions, approve or reject papers with reasons, manage user accounts, view analytics dashboards, and inspect the full audit trail of moderation actions.

2. SYSTEM REQUIREMENTS

This chapter specifies the hardware, software, and tooling required both to develop LUMINA and to run it in a typical deployment. Because LUMINA is a web application, the requirements are divided between the server that hosts the application and the client devices used to access it.

2.1 Hardware Requirements

The hardware needs of LUMINA are modest, which is one of the advantages of a lightweight Node.js backend paired with a relational database. The recommended minimum configurations are given below.

Component	Development Machine	Production Server
Processor	Intel i3 / equivalent, dual-core	2 vCPU minimum, 4 vCPU recommended
RAM	4 GB minimum, 8 GB recommended	4 GB minimum, 8 GB recommended
Storage	10 GB free (code + dependencies)	50 GB+ SSD (grows with uploaded PDFs)
Network	Broadband internet	Stable public connection / static IP
Display	1366 x 768 or higher	Not required (headless)

On the client side, the only hardware requirement is any device – desktop, laptop, tablet, or smartphone – capable of running a modern web browser. Because the interface is fully responsive, no special screen size is mandatory.

2.2 Software Requirements

The software stack is built entirely on free and open-source components, which keeps the cost of ownership low and avoids vendor lock-in.

Layer	Software	Version
Runtime	Node.js	18.x LTS or higher
Package manager	npm	9.x or higher
Database	MySQL	8.0 or higher
Frontend library	React	18.x
Build tool	Vite	5.x
Operating system	Windows / Linux / macOS	Any current version
Browser (client)	Chrome / Firefox / Edge / Safari	Latest two versions

2.3 Development Tools and Frameworks

The following tools and libraries were used during the development of LUMINA. They are grouped by the role they play in the project.

Backend libraries

- express – the web framework that defines routes and middleware.
- mysql2/promise – a promise-based MySQL driver with connection pooling and prepared statements.
- jsonwebtoken – signs and verifies JWT session tokens.
- bcryptjs – hashes and compares passwords using the bcrypt algorithm.
- multer – middleware that handles multipart file uploads for PDFs and ZIPs.
- pdf-parse – extracts text and metadata from PDF files.
- nodemailer – sends SMTP email notifications.
- dotenv – loads environment variables from a .env file.
- cors – enables and restricts cross-origin requests from the frontend.

Frontend libraries

- react and react-dom – the core UI library.
- react-router-dom (v6) – client-side routing and protected routes.
- axios – the HTTP client used to call the backend API, configured with a JWT interceptor.
- react-dropzone – drag-and-drop file selection in the Upload Wizard.
- recharts – charts used in the admin analytics dashboard.

Tooling

- Git and GitHub – version control and collaboration.
- Visual Studio Code – primary code editor.
- Postman – manual testing of API endpoints.
- MySQL Workbench – database design and inspection.

Functional requirements

Functional requirements describe what the system must do. The core functional requirements of LUMINA are:

- The system shall allow a visitor to register an account and log in securely.
- The system shall allow a logged-in researcher to upload a paper with an optional code archive through a multi-step wizard.

- The system shall automatically extract metadata from an uploaded PDF and detect duplicate files.
- The system shall hold every submission in a pending state until an administrator approves or rejects it.
- The system shall allow administrators to log in with two-factor authentication and to approve, reject, or delete papers with a recorded reason.
- The system shall allow any visitor to search and filter approved public papers.
- The system shall enforce public, restricted, and private visibility on every paper and support access requests and time-limited share links.
- The system shall award contributor badges and rank trending papers by genuine weekly views.
- The system shall notify users of approval, rejection, and access events in-app and, where configured, by email and Telegram.

Non-functional requirements

Non-functional requirements describe how well the system must perform its functions. The principal non-functional requirements are:

Quality	Requirement
Security	Passwords hashed with bcrypt; JWT sessions; admin 2FA; parameterised queries; HTTPS in production.
Performance	Common pages respond within a second under normal load using indexed queries and a connection pool.
Usability	Clean, responsive interface that works on desktop, tablet, and mobile with no training required.
Reliability	Soft deletes and an immutable audit log ensure no action is silently lost or unrecoverable.
Maintainability	Modular, layered codebase so new features can be added with minimal disruption.
Portability	Runs on Windows, Linux, or macOS; configuration entirely through environment variables.
Scalability	Stateless API can be replicated horizontally behind a reverse proxy.

3. TECHNOLOGY STACK

LUMINA follows a classic three-tier architecture: a presentation tier (the React frontend), an application tier (the Node.js / Express API), and a data tier (the MySQL database). This chapter describes each technology in the stack and explains why it was selected.

3.1 Front-end (React.js and Vite)

The user interface is built with React 18, a declarative, component-based JavaScript library. React was chosen because its component model maps naturally onto the repeating visual elements of LUMINA – paper cards, navigation bars, search inputs, and form steps can each be expressed as a reusable component. React's virtual DOM efficiently updates only the parts of the page that change, which keeps the interface responsive even as the list of papers grows.

Vite is used as the build tool and development server. Compared to older bundlers, Vite offers near-instant hot module replacement during development and produces a highly optimised, tree-shaken bundle for production. React Router v6 manages client-side navigation, allowing LUMINA to behave as a single-page application where moving between pages does not trigger a full reload. The frontend is styled with a custom CSS design system rather than a heavyweight UI framework, which keeps the bundle small and the visual identity unique.

Key React concepts applied

Concept	Where it is applied in LUMINA
Props	PaperCard, SearchBar, BadgeIcon, CategoryZone receive data from parents
Lifting state up	AllPapers holds shared filter/search state used by children
Context API	AuthContext provides global user and token state
Custom hooks	useAdminActions encapsulates approve/reject/delete logic
Controlled components	All forms (Login, Register, Upload Wizard, Profile)
Conditional rendering	Role-based UI, access states, loading and error states
useEffect / useState	Data fetching, localStorage sync, admin polling, form state
useParams / useSearchParams	Dynamic paper routes and query-string filters

A representative component is the PaperCard used throughout the browse and trending pages. It receives a paper object as a prop and renders its title, authors, category badge, and view count, and navigates to the detail page when clicked – a small, self-contained, reusable unit typical of the component-based approach.

```
// components/PaperCard.jsx
function PaperCard({ paper }) {
  const navigate = useNavigate();
```

```

return (
  <div className="paper-card" onClick={() => navigate(`/papers/${paper.id}`)}>
    <span className={` badge ${paper.category}`}>{paper.category}</span>
    <h3>{paper.title}</h3>
    <p className="authors">{paper.authors.join(', ')}</p>
    <div className="meta">
      <span>{paper.year}</span>
      <span>{paper.view_count} views</span>
    </div>
  </div>
);
}

```

3.2 Back-end (Node.js and Express.js)

The backend is a RESTful API built on Node.js, a JavaScript runtime that executes server-side code on the V8 engine. Node.js was selected because it allows the entire project to be written in a single language – JavaScript – across both client and server, which simplifies development and knowledge sharing within the team. Its non-blocking, event-driven model is well suited to an I/O-heavy application that spends much of its time reading files and querying the database.

Express.js sits on top of Node and provides a clean, minimal framework for defining routes, attaching middleware, and structuring the application. The backend is organised into a conventional layered structure: routes define the URL surface, controllers contain the business logic, middleware handles cross-cutting concerns such as authentication and file uploads, and utility modules contain reusable helpers like the PDF parser and the badge updater. The server listens on port 5000 and exposes its API under the /api prefix.

```

// server.js (simplified entry point)
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors({ origin: process.env.CLIENT_URL }));
app.use(express.json());
app.use('/api/auth', require('./routes/auth'));
app.use('/api/papers', require('./routes/papers'));
app.use('/api/admin', require('./routes/admin'));
app.use('/api/users', require('./routes/users'));
app.use('/api/categories', require('./routes/categories'));

app.listen(5000, () => console.log('LUMINA API on port 5000'));

```

Within each route file, Express maps HTTP verbs and paths to controller functions and chains the appropriate middleware. The papers route below shows how public endpoints, authenticated endpoints, and file-upload endpoints sit side by side, each protected exactly as much as it needs to be.

```

// routes/papers.js
const router = require('express').Router();

```

```

const ctrl = require('../controllers/paperController');
const auth = require('../middleware/auth');
const upload = require('../middleware/upload'); // Multer

router.get('/', ctrl.list); // public
router.get('/trending', ctrl.trending); // public
router.get('/:id', ctrl.detail); // optional auth
router.post('/upload-temp', auth, upload.single('file'), ctrl.uploadTemp);
router.post('/extract', auth, ctrl.extract);
router.post('/submit', auth, ctrl.submit);
router.post('/:id/request-access', auth, ctrl.requestAccess);
router.post('/:id/share-link', auth, ctrl.shareLink);
module.exports = router;

```

3.3 Database (MySQL)

LUMINA stores all structured data in a MySQL relational database. A relational database was chosen because the domain is naturally relational: users author papers, papers belong to categories, papers receive views and access requests, and administrators record moderation actions. These relationships are expressed cleanly with foreign keys, and MySQL enforces referential integrity automatically.

The application connects to MySQL through the `mysql2/promise` driver, which exposes a promise-based interface and supports a connection pool. A pool of ten reusable connections is created at startup, which avoids the overhead of opening a new connection for every request and lets the server handle concurrent traffic efficiently. Every query is executed as a parameterised prepared statement, which is the primary defence against SQL injection. The schema comprises nine tables and includes a FULLTEXT index on the `research_papers` table to power keyword search.

```

// config/db.js
const mysql = require('mysql2/promise');
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
});
module.exports = pool;

```

Operations that change several tables at once – such as approving a paper, which updates the paper, writes an audit log, recalculates a badge, and inserts a notification – are wrapped in a transaction. A dedicated connection is taken from the pool, the statements run inside a `begin/commit` block, and any error triggers a rollback so the database is never left in a half-updated state. This use of transactions is what guarantees that a moderation action is all-or-nothing, which is essential for keeping the audit trail and the published state consistent with one another.

3.4 Version Control (Git and GitHub)

The project source code is managed with Git and hosted on GitHub. Git provides a complete history of every change, allowing the team to experiment on feature branches and merge work safely. GitHub serves as the shared remote repository, enabling collaboration between team members, code review through pull requests, and a single source of truth for the latest code. A .gitignore file excludes node_modules, the .env file, and uploaded user content so that secrets and large binaries are never committed.

3.5 APIs and External Services

LUMINA integrates with a small number of external services to deliver features that go beyond the core database:

Telegram Bot API: Used to deliver two-factor one-time passwords to administrators during login. When an admin enters a valid username and password, a six-digit code is generated, stored with a five-minute expiry, and pushed to the admin's Telegram chat through the bot.

SMTP Email (Nodemailer): Used to send email notifications to researchers when their paper is approved, rejected, or deleted. Email delivery is optional and activates only when SMTP credentials are configured.

arXiv Integration: A seed utility can populate the database with real, openly licensed papers fetched from arXiv, which is useful for demonstrations and for testing search and browsing with realistic data.

3.6 Justification of Technology Choices

Each major technology was selected after weighing it against credible alternatives. The table below summarises why the chosen stack was preferred for a project of this nature.

Layer	Chosen	Why over alternatives
Frontend	React + Vite	Component reuse and a huge ecosystem; preferred over plain JS (no structure) and Angular (heavier learning curve).
Backend	Node.js + Express	Same language as the frontend; lightweight and fast for I/O-bound work; preferred over PHP and Java/Spring for team familiarity.
Database	MySQL	Strong relational integrity for a clearly relational domain; preferred over MongoDB, whose flexible documents would weaken the many foreign-key relationships here.
Auth	JWT	Stateless tokens scale without server-side sessions; preferred over cookie sessions for a decoupled SPA + API.
2FA	Telegram OTP	Free and reliable; preferred over paid SMS gateways.

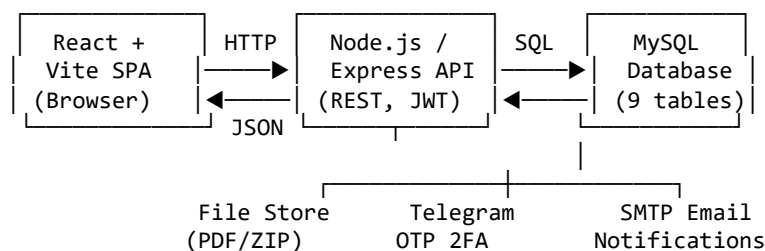
The guiding principle was to choose a single-language, open-source stack that the team could be fully productive in, that imposed strong structure where the domain demanded it (the relational database), and that avoided any paid dependency. The result is a stack that is both pedagogically valuable and genuinely production-appropriate for an institutional repository.

4. SYSTEM ARCHITECTURE

This chapter describes how the major parts of LUMINA fit together. The system follows a three-tier client-server architecture in which a browser-based client communicates with a stateless API server, which in turn reads from and writes to a relational database and a file store.

4.1 High-Level Architecture Diagram

The diagram below shows the flow of a typical request through the system. The React client issues an HTTP request carrying a JWT; the Express API authenticates the token, applies business logic in a controller, and queries MySQL; binary files such as PDFs are written to and served from the server file system; and external services handle email and Telegram delivery.



4.2 Description of Each Layer

Presentation layer (Frontend)

The presentation layer is the React single-page application that runs inside the user's browser. It is responsible for rendering all screens, capturing user input through controlled form components, managing global authentication state through a React Context, and communicating with the backend exclusively through the REST API. It never talks to the database directly. The JWT returned at login is stored in localStorage and attached to every outgoing request by an Axios interceptor, so the server can identify and authorise the user.

Application layer (Backend)

The application layer is the Express API server. It is stateless: it keeps no session in memory, instead trusting the signed JWT presented on each request. Incoming requests pass through middleware that verifies the token and, for admin routes, checks the user's role. Controllers then execute the business logic – validating input, hashing files, moving uploads into permanent storage, recalculating badges, writing audit logs, and dispatching notifications. This layer is the only part of the system permitted to read or write the database and the file store.

Data layer (Database and File Store)

The data layer consists of the MySQL database and the server file system. MySQL holds all structured data – users, papers, categories, notifications, views, access requests, share links, and

audit logs. Large binary files are not stored in the database; instead the PDF and ZIP files are written to a dated directory structure on disk (uploads/papers/YYYY/MM/ and uploads/code/YYYY/MM/), and only their file paths and SHA-256 hashes are recorded in the database. This separation keeps the database lean and makes file serving efficient.

Project structure

The codebase is organised into a clear frontend and backend separation, each with its own conventional folder layout. The principal building blocks are listed below.

Frontend (React) area	Responsibility
pages/	Top-level screens: Home, AllPapers, PaperDetail, UploadWizard, Login, Register, Dashboard, Admin/*
components/	Reusable UI: PaperCard, SearchBar, FilterBar, BadgeIcon, Navbar
context/	AuthContext for global user and token state
api/	Configured Axios client and grouped endpoint helpers
hooks/	Custom hooks such as useAdminActions

Backend (Node) area	Responsibility
routes/	URL surface: auth, papers, admin, users, categories
controllers/	Business logic for each module
middleware/	auth, adminMiddleware, Multer upload
utils/	PDF parser, badge updater, hashing, Telegram, email
config/	Database pool and environment loading

4.3 Deployment Architecture

In a production deployment, the three tiers are separated for scalability and security. The React application is compiled by Vite into a set of static files (HTML, CSS, JavaScript) that can be served by any static host or CDN. The Node.js API runs as a long-lived process, typically behind a process manager such as PM2 and a reverse proxy such as Nginx that terminates HTTPS and forwards API traffic to port 5000. The MySQL database runs as a managed service or on a dedicated instance, accessible only from the API server. Uploaded files reside on a persistent volume attached to the API server. Environment-specific configuration – database credentials, the JWT secret, SMTP and Telegram tokens – is supplied through environment variables and never committed to source control.

4.4 Request Lifecycle

Understanding a single request end-to-end clarifies how the layers cooperate. Consider a logged-in researcher opening a restricted paper:

1. The browser issues GET /api/papers/:id. The Axios interceptor attaches the stored JWT as a Bearer token.
2. Nginx receives the HTTPS request, terminates TLS, and forwards it to the Node process on port 5000.
3. Express routes the request to the paper controller after authMiddleware verifies the token signature and attaches req.user.
4. The controller loads the paper, evaluates its visibility against the requester's identity and grants, and either returns the record or responds 403.
5. If access is allowed, a paper_views row is recorded for the current ISO week (ignored if one already exists), and the JSON response travels back through Nginx to the browser, where React renders the detail view.

The entire exchange is stateless on the server: nothing about the user is held in memory between requests, which means any instance of the API can serve any request and the system can be scaled horizontally simply by running more instances behind the proxy.

4.5 Security Architecture

Security is layered rather than concentrated in a single mechanism, following the principle of defence in depth. At the transport layer, all traffic is encrypted with HTTPS. At the authentication layer, passwords are never stored in plain text – they are hashed with bcrypt – and sessions are represented by signed JWTs that cannot be forged without the server secret. At the authorization layer, middleware enforces role checks, and every access-controlled resource re-evaluates the requester's rights on the server rather than trusting the client. At the data layer, every query is a parameterised prepared statement, which closes off SQL-injection attacks, and cross-origin requests are restricted by a CORS policy to the known frontend origin. Finally, administrator accounts – the most powerful in the system – are protected by a second factor delivered over Telegram, so that a leaked password alone is not enough to gain administrative control.

5. DESIGN

This chapter presents the data design of LUMINA. Because the application is fundamentally a system for storing and relating records of papers, users, and moderation events, the database schema is the heart of its design.

5.1 ER Diagram / Database Schema

The database consists of nine tables. The central entity is `research_papers`, which is connected to users (the author), categories, and a number of supporting tables that record views, access requests, share links, notifications, and admin actions. The relationships are summarised below.

Entity relationships

- `users` → `research_papers` (one author has many papers)
- `users` → `access_requests`, `paper_views`, `notifications` (one user, many of each)
- `categories` → `research_papers` (one category, many papers)
- `research_papers` → `notifications`, `access_requests`, `paper_views`, `private_share_links`, `admin_logs` (one paper, many of each)
- `admins` → `admin_logs` and `research_papers.approved_by` (one admin, many actions)

Table: users

Column	Type	Notes
<code>id</code>	CHAR(36) PK	UUID
<code>name</code>	VARCHAR(100)	NOT NULL
<code>email</code>	VARCHAR(150)	UNIQUE, NOT NULL
<code>institution</code>	VARCHAR(200)	
<code>research_role</code>	VARCHAR(100)	
<code>bio</code>	TEXT	
<code>scholar_links</code>	JSON	Google Scholar, ORCID, etc.
<code>password_hash</code>	VARCHAR(255)	bcrypt, 10 salt rounds
<code>badge_level</code>	ENUM	newcomer / bronze / silver / gold / diamond
<code>approved_count</code>	INT	DEFAULT 0
<code>role</code>	VARCHAR(20)	DEFAULT 'user'
<code>is_active</code>	BOOLEAN	Soft-delete flag, DEFAULT TRUE
<code>created_at</code>	TIMESTAMP	

Table: research_papers (central entity)

Column	Type	Notes
id	CHAR(36) PK	UUID
user_id	CHAR(36) FK	Author → users
title	VARCHAR(500)	
authors / keywords	JSON	Arrays
abstract	TEXT	
doi / journal / year	mixed	Bibliographic metadata
category_id	INT FK	→ categories
visibility	ENUM	public / private / restricted
license	ENUM	open_access / copyright_reserved / creative_commons
status	ENUM	pending / approved / rejected
file_path / file_hash	VARCHAR	Path + SHA-256 of PDF
code_path / code_hash	VARCHAR	Path + SHA-256 of ZIP
view_count	INT	DEFAULT 0
is_deleted	BOOLEAN	Soft delete
approved_by	INT FK	→ admins

Indexes: a FULLTEXT index spans (title, abstract, journal, sub_category) to support keyword search, and conventional indexes exist on the foreign keys user_id, category_id, and approved_by as well as on status to make moderation queries fast.

Table: admins

Column	Type	Notes
id	INT AI PK	Auto-increment
username	VARCHAR(50)	UNIQUE
password_hash	VARCHAR(255)	bcrypt
telegram_chat_id	VARCHAR(50)	For OTP delivery
otp_code	VARCHAR(10)	6-digit, single-use
otp_expires_at	DATETIME	5-minute window
created_at	TIMESTAMP	Account creation time

Table: categories

Column	Type	Notes
id	INT AI PK	Auto-increment
name	VARCHAR(100)	AI, ML, Web Dev, Biotech, Physics, etc.
sub_category	VARCHAR(100)	Optional refinement

Column	Type	Notes
icon_color	VARCHAR(30)	UI accent colour
description	TEXT	Short description
paper_count	INT	Auto-updated on submission

Table: notifications

Column	Type	Notes
id	INT AI PK	Auto-increment
user_id	CHAR(36) FK	→ users
paper_id	CHAR(36) FK	→ research_papers
type	ENUM	approved / rejected / deleted / access_request / access_granted
message	TEXT	Human-readable text
is_read	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP	When raised

Table: access_requests

Column	Type	Notes
id	INT AI PK	Auto-increment
requester_id	CHAR(36) FK	→ users
paper_id	CHAR(36) FK	→ research_papers
status	ENUM	pending / granted / denied
requested_at	TIMESTAMP	When requested
UNIQUE	(requester_id, paper_id)	One request per user per paper

Table: paper_views

Column	Type	Notes
id	INT AI PK	Auto-increment
user_id	CHAR(36) FK	Nullable (guest views)
paper_id	CHAR(36) FK	→ research_papers
viewed_at	TIMESTAMP	When viewed
view_week	VARCHAR(10)	YYYY-WW format
UNIQUE	(user_id, paper_id, view_week)	Prevents duplicate weekly views

Table: private_share_links

Column	Type	Notes
id	INT AI PK	Auto-increment
paper_id	CHAR(36) FK	→ research_papers
token	VARCHAR(64)	UNIQUE random share token
expires_at	DATETIME	7 days from creation
created_at	TIMESTAMP	When generated

Table: admin_logs

Column	Type	Notes
id	INT AI PK	Auto-increment
admin_id	INT FK	→ admins
paper_id	CHAR(36) FK	→ research_papers
action	ENUM	approve / reject / delete
reason	TEXT	Justification for the action
created_at	TIMESTAMP	When performed

Normalisation and Design Rationale

The schema is normalised to the third normal form. Each table represents a single, well-defined entity, every non-key attribute depends on the whole primary key, and there are no transitive dependencies. For example, category information is held once in the categories table and referenced by foreign key from research_papers rather than being duplicated on every paper row, which prevents update anomalies. Many-to-many style relationships, such as the views between users and papers, are resolved through dedicated junction-style tables (paper_views, access_requests) that also carry their own attributes.

A small number of denormalised, derived values are kept deliberately for performance: categories.paper_count and users.approved_count are maintained by the application whenever the underlying rows change, so that frequently displayed totals do not require an expensive aggregate query on every page load. UUID primary keys (CHAR(36)) are used for users and papers so that identifiers are globally unique, non-sequential, and safe to expose in URLs, while the smaller auxiliary tables use simple auto-increment integers.

Schema Definition (DDL)

For completeness, the core of the schema is expressed below as MySQL data-definition statements. These show the exact types, keys, defaults, and the FULLTEXT index that the application relies upon.

```

CREATE TABLE users (
  id          CHAR(36) PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE NOT NULL,
  institution  VARCHAR(200),
  research_role VARCHAR(100),
  bio         TEXT,
  scholar_links JSON,
  password_hash VARCHAR(255) NOT NULL,
  badge_level  ENUM('newcomer','bronze','silver','gold','diamond')
              DEFAULT 'newcomer',
  approved_count INT DEFAULT 0,
  role         VARCHAR(20) DEFAULT 'user',
  is_active    BOOLEAN DEFAULT TRUE,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE research_papers (
  id          CHAR(36) PRIMARY KEY,
  user_id     CHAR(36) NOT NULL,
  title       VARCHAR(500) NOT NULL,
  authors     JSON,
  keywords    JSON,
  abstract    TEXT,
  doi         VARCHAR(100),
  journal     VARCHAR(200),
  year        INT,
  category_id INT,
  sub_category VARCHAR(100),
  visibility  ENUM('public','private','restricted') DEFAULT 'public',
  license     ENUM('open_access','copyright_reserved','creative_commons'),
  status      ENUM('pending','approved','rejected') DEFAULT 'pending',
  file_path   VARCHAR(255),
  file_hash   CHAR(64),
  code_path   VARCHAR(255),
  code_hash   CHAR(64),
  view_count  INT DEFAULT 0,
  is_deleted  BOOLEAN DEFAULT FALSE,
  approved_by INT,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (category_id) REFERENCES categories(id),
  FOREIGN KEY (approved_by) REFERENCES admins(id),
  FULLTEXT KEY ft_search (title, abstract, journal, sub_category)
);

CREATE TABLE paper_views (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     CHAR(36),
  paper_id    CHAR(36) NOT NULL,
  viewed_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  view_week   VARCHAR(10) NOT NULL,
  UNIQUE KEY uq_weekly (user_id, paper_id, view_week),
  FOREIGN KEY (paper_id) REFERENCES research_papers(id)
);

CREATE TABLE access_requests (

```

```

    id            INT AUTO_INCREMENT PRIMARY KEY,
    requester_id  CHAR(36) NOT NULL,
    paper_id      CHAR(36) NOT NULL,
    status        ENUM('pending','granted','denied') DEFAULT 'pending',
    requested_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_request (requester_id, paper_id),
    FOREIGN KEY (requester_id) REFERENCES users(id),
    FOREIGN KEY (paper_id) REFERENCES research_papers(id)
);

CREATE TABLE private_share_links (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    paper_id      CHAR(36) NOT NULL,
    token         VARCHAR(64) UNIQUE NOT NULL,
    expires_at    DATETIME NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (paper_id) REFERENCES research_papers(id)
);

CREATE TABLE admin_logs (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    admin_id      INT NOT NULL,
    paper_id      CHAR(36),
    action        ENUM('approve','reject','delete') NOT NULL,
    reason        TEXT,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (admin_id) REFERENCES admins(id)
);

```

6. IMPLEMENTATION

This chapter explains how the major modules of LUMINA were implemented, moving from the overall module breakdown to the specifics of frontend logic, API integration, and the authentication subsystem.

6.1 Module-wise Implementation

LUMINA is decomposed into a set of cohesive modules, each owning a clear responsibility on both the frontend and the backend.

Authentication module: Handles registration, user login, and the two-step admin login with Telegram OTP. On the backend it lives in `authController.js`; on the frontend it is exposed through `AuthContext` and the Login and Register pages.

Paper module: The largest module. It manages temporary uploads, PDF metadata extraction, final submission, listing with filters, trending, the paper detail view, access requests, and share links. The backend logic lives in `paperController.js` and the frontend in the Upload Wizard, AllPapers, and PaperDetail components.

Admin module: Provides the analytics dashboard, the pending-paper queue, approve / reject / delete actions, user administration, and the audit-log viewer. Implemented in adminController.js and the Admin/* pages.

User module: Serves the researcher dashboard – profile data, the user's own papers, notifications, and access requests – and handles profile updates.

Category module: Lists subject categories with their live paper counts, used by both the browse filters and the upload form.

As an illustration of the controller logic, the approval handler in the admin module shows how a single action coordinates several side effects – updating status, writing the audit log, recalculating the badge, and notifying the author – all within one database transaction so that the operation either fully succeeds or fully rolls back.

```
// adminController.js (approve)
async function approvePaper(req, res) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    await conn.query(
      "UPDATE research_papers SET status='approved', approved_by=? WHERE id=?",
      [req.user.id, req.params.id]);
    await conn.query(
      'INSERT INTO admin_logs (admin_id, paper_id, action, reason) VALUES (?, ?, ?, ?)',
      [req.user.id, req.params.id, 'approve', req.body.reason || 'Meets guidelines']);
    await recalcBadge(conn, paper.user_id);
    await notify(conn, paper.user_id, req.params.id, 'approved');
    await conn.commit();
    res.json({ message: 'Paper approved' });
  } catch (e) { await conn.rollback(); res.status(500).json({ error: e.message }); }
  finally { conn.release(); }
}
```

The recalcBadge and notify helpers used above are small, focused utilities. The badge helper counts the author's approved papers and writes back the appropriate level, while the notify helper inserts a notification row and, where configured, dispatches an email and a Telegram message.

```
// utils/badge.js
async function recalcBadge(conn, userId) {
  const [{ n }] = await conn.query(
    "SELECT COUNT(*) n FROM research_papers" +
    " WHERE user_id=? AND status='approved' AND is_deleted=FALSE",
    [userId]);
  const level = n >= 30 ? 'diamond' : n >= 15 ? 'gold'
    : n >= 5 ? 'silver' : n >= 1 ? 'bronze' : 'newcomer';
  await conn.query(
    'UPDATE users SET approved_count=?, badge_level=? WHERE id=?',
    [n, level, userId]);
}
```

```
// utils/notify.js
async function notify(conn, userId, paperId, type) {
  const messages = {
    approved: 'Your paper has been approved and published.',
    rejected: 'Your paper was not approved. See the reason in your dashboard.',
    deleted: 'Your paper has been removed by an administrator.',
  };
  await conn.query(
    'INSERT INTO notifications (user_id, paper_id, type, message) VALUES (?, ?, ?, ?)',
    [userId, paperId, type, messages[type]];
  if (mailEnabled) await sendEmail(userId, messages[type]);
  if (telegramEnabled) await sendTelegramToUser(userId, messages[type]);
}
```

6.2 Front-end Logic (UI Rendering and State Management)

The frontend is a single-page application. At its core is a small set of global concerns – who the current user is and what token they hold – which are managed through React's Context API rather than being passed down through many layers of props. The AuthContext provider exposes the current user, the token, a saveSession function, and a logout function, and it persists both values to localStorage so that a page refresh does not log the user out.

```
// context/AuthContext.jsx (essence)
const AuthContext = createContext();
export function AuthProvider({ children }) {
  const [user, setUser] = useState(JSON.parse(localStorage.getItem('user')));
  const [token, setToken] = useState(localStorage.getItem('token'));
  const saveSession = (tk, usr) => {
    localStorage.setItem('token', tk);
    localStorage.setItem('user', JSON.stringify(usr));
    setToken(tk); setUser(usr);
  };
  const logout = () => { localStorage.clear(); setUser(null); setToken(null); };
  return <AuthContext.Provider value={{ user, token, saveSession, logout }}>
    {children}
  </AuthContext.Provider>;
}
export const useAuth = () => useContext(AuthContext);
```

Individual screens manage their own local state with useState – form fields, pagination, loading flags, and modal toggles – and fetch data with useEffect when they mount. Pages that share state, such as AllPapers and its child SearchBar and filter components, lift that state up to the common parent so a single source of truth drives the displayed results. Routes are declared in main.jsx, and a ProtectedRoute wrapper redirects unauthenticated users to the login page and non-admins away from admin pages.

```
// main.jsx (protected route)
function ProtectedRoute({ children, adminOnly }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" />;
  if (adminOnly && user.role !== 'admin') return <Navigate to="/" />;
  return children;
}
```

```
}
```

The Upload Wizard is the most state-intensive screen in the application. It keeps a single form object and a current-step index in component state, advancing and retreating through the seven steps while preserving everything the user has entered. Each field is a controlled input bound to this object, so the review step can simply read back the accumulated values before submission.

```
// pages/UploadWizard.jsx (state essence)
const [step, setStep] = useState(1);
const [form, setForm] = useState({
  title: '', authors: [], abstract: '', keywords: [],
  doi: '', journal: '', year: '', categoryId: null,
  visibility: 'public', license: 'open_access',
  filePath: null, codePath: null,
});
const update = (field, value) => setForm(f => ({ ...f, [field]: value }));
const next = () => setStep(s => Math.min(7, s + 1));
const back = () => setStep(s => Math.max(1, s - 1));
```

Data-fetching screens such as AllPapers load their content with `useEffect` when they mount and whenever the active filters change, calling the API and storing the returned papers in local state. A loading flag drives a spinner while the request is in flight, and any error is captured and shown to the user rather than failing silently.

6.3 API Integration

All communication between the frontend and backend flows through a single configured `Axios` instance. A request interceptor reads the JWT from `localStorage` and attaches it as a Bearer token to the `Authorization` header of every outgoing request, so individual API calls never have to worry about authentication.

```
// api/client.js
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
export default api;
```

API calls are grouped by domain in an endpoints module, giving the rest of the application a clean, named interface such as `paperApi.submit(...)` or `adminApi.approve(id)` instead of scattering raw URLs throughout the components. The complete REST surface exposed by the backend is summarised below.

Method	Route	Auth	Action
POST	/api/auth/register	None	Register new user
POST	/api/auth/login	None	User login

Method	Route	Auth	Action
POST	/api/auth/admin/login	None	Admin step 1 (send OTP)
POST	/api/auth/admin/verify-otp	None	Admin step 2 (verify OTP)
GET	/api/papers	None	List approved papers (filters)
GET	/api/papers/trending	None	Trending by weekly views
GET	/api/papers/:id	Optional	Paper detail (access-controlled)
POST	/api/papers/upload-temp	Required	Upload PDF/ZIP to staging
POST	/api/papers/extract	Required	Extract PDF metadata
POST	/api/papers/submit	Required	Submit paper for review
POST	/api/papers/:id/request-access	Required	Request access to private paper
POST	/api/papers/:id/share-link	Required	Generate 7-day share link
GET	/api/admin/dashboard	Admin	Analytics and recent activity
PATCH	/api/admin/papers/:id/approve	Admin	Approve a paper
PATCH	/api/admin/papers/:id/reject	Admin	Reject with reason
DELETE	/api/admin/papers/:id	Admin	Delete with reason
GET	/api/users/me	Required	Profile + papers + notifications
GET	/api/categories	None	List categories with counts

Sample request and response

To make the API contract concrete, a representative exchange is shown below. The client submits a paper for review and receives the identifier of the newly created pending record.

```
POST /api/papers/submit
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "title": "A Survey of Edge AI for Road Defect Detection",
  "authors": ["N. Lokesh Reddy", "R. Rishik Rao"],
  "abstract": "This paper reviews lightweight detection models ...",
  "keywords": ["edge ai", "yolo", "road defects"],
  "year": 2026,
  "categoryId": 2,
  "visibility": "public",
  "license": "open_access",
  "filePath": "tmp/ab12cd34.pdf",
  "fileHash": "9f86d081884c7d659a2feaa0c55ad015..."
}
```

```

}
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "6b1e7c2a-9f4d-4a3b-8c11-2d5e9a7f0b33",
  "message": "Submitted for review"
}

```

A typical error response follows the same JSON convention, returning an appropriate status code and a single descriptive error field, which keeps client-side error handling uniform across every endpoint.

```

HTTP/1.1 401 Unauthorized
{ "error": "Invalid token" }

```

6.4 Authentication and Authorization

Authentication establishes who a user is; authorization decides what they are allowed to do. LUMINA implements both with a JWT-based scheme. When a user registers or logs in, the server hashes (on registration) or verifies (on login) the password with bcrypt, then signs a JWT containing the user's id, name, email, and role. The token has a seven-day expiry and is returned to the client, which stores it and presents it on subsequent requests.

```

// authController.js (login essence)
const ok = await bcrypt.compare(password, user.password_hash);
if (!ok) return res.status(401).json({ error: 'Invalid credentials' });
const token = jwt.sign(
  { id: user.id, name: user.name, email: user.email, role: 'user' },
  process.env.JWT_SECRET, { expiresIn: '7d' });
res.json({ token, user });

```

On every protected request, authMiddleware reads the Bearer token, verifies its signature with the secret, and attaches the decoded payload to req.user. For administrator-only routes, a second adminMiddleware checks that req.user.role is 'admin' and otherwise responds with 403 Forbidden. Administrator login is hardened with two-factor authentication: after the password check, a six-digit OTP is generated, stored with a five-minute expiry, and sent over Telegram; only after the correct, unexpired OTP is submitted does the server issue an admin JWT.

```

// middleware/auth.js
function authMiddleware(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch { return res.status(401).json({ error: 'Invalid token' }); }
}
function adminMiddleware(req, res, next) {

```

```

if (req.user?.role !== 'admin')
  return res.status(403).json({ error: 'Admin only' });
next();
}

```

The two-factor step for administrators is implemented by generating a random six-digit code, persisting it with a short expiry, and delivering it over the Telegram Bot API. Verification simply checks the submitted code against the stored one and confirms it has not expired.

```

// admin OTP generation
const otp = ('' + Math.floor(100000 + Math.random() * 900000));
const expires = new Date(Date.now() + 5 * 60 * 1000); // 5 minutes
await pool.query('UPDATE admins SET otp_code=?, otp_expires_at=? WHERE id=?',
  [otp, expires, admin.id]);
await sendTelegram(admin.telegram_chat_id, `LUMINA admin OTP: ${otp}`);

```

Role-based access matrix

Action	Guest	Researcher	Admin
View public papers	Yes	Yes	Yes
View restricted papers	No	Yes	Yes
View private papers	No	Owner / granted	Yes
Upload papers	No	Yes	No
Request access	No	Yes	No
Approve / reject papers	No	No	Yes
Manage users	No	No	Yes
View analytics	No	No	Yes

End-to-end Upload Data Flow

To illustrate how the modules cooperate, the complete path a paper travels from selection to publication is traced below. This single flow touches the frontend, the API, the database, the file store, and the external notification services, and is therefore the best example of the system working as a whole.

6. The researcher drags a PDF onto the dropzone in step one of the Upload Wizard. The file is sent to POST /api/papers/upload-temp as multipart form data, handled by Multer.
7. The server writes the file to a temporary staging directory, reads its bytes, and computes a SHA-256 hash.
8. The hash is checked against research_papers.file_hash. If a duplicate is found, the user is warned before they waste effort filling in the rest of the form.
9. The client then calls POST /api/papers/extract, which runs pdf-parse over the staged file and returns a best-effort title, author list, and abstract.

10. The wizard pre-fills the metadata fields with the extracted values; the researcher reviews and corrects them, adds keywords, DOI, journal and year, and chooses category, visibility, and license.
11. If source code is attached, the ZIP is uploaded and hashed in the same way as the PDF.
12. On final submission the client calls POST /api/papers/submit with all metadata and the staged file references.
13. The server moves the staged files into permanent dated storage (uploads/papers/YYYY/MM/), inserts a research_papers row with status 'pending', and increments the category paper_count.
14. The submission appears in the administrator's pending queue. The admin reviews and approves it.
15. Approval flips the status to 'approved', records the action in admin_logs, recalculates the author's badge, raises a notification, and optionally emails the author. The paper now appears in public listings and search.

This flow demonstrates the separation of concerns that runs throughout LUMINA: the frontend only collects and displays data, the API enforces every rule and performs every side effect, and the database and file store passively hold state. No business rule is ever trusted to the client.

The submit handler that completes this flow validates the payload, moves the staged files into permanent storage, and creates the pending record inside a transaction:

```
// paperController.js (submit)
async function submit(req, res) {
  const id = uuidv4();
  const finalPdf = moveToPermanent(req.body.filePath, 'papers');
  const finalZip = req.body.codePath
    ? moveToPermanent(req.body.codePath, 'code') : null;
  await pool.query(
    `INSERT INTO research_papers
      (id,user_id,title,authors,keywords,abstract,doi,journal,year,
       category_id,visibility,license,status,file_path,file_hash,
       code_path,code_hash)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
    [id, req.user.id, req.body.title, JSON.stringify(req.body.authors),
     JSON.stringify(req.body.keywords), req.body.abstract, req.body.doi,
     req.body.journal, req.body.year, req.body.categoryId,
     req.body.visibility, req.body.license, finalPdf, req.body.fileHash,
     finalZip, req.body.codeHash]);
  await bumpCategoryCount(req.body.categoryId);
  res.status(201).json({ id, message: 'Submitted for review' });
}
```

The access decision that guards every private paper is centralised so that ownership, explicit grants, and valid share tokens are all considered together in one place:

```
// paperController.js (access check)
async function canAccess(paper, user, shareToken) {
  if (paper.visibility === 'public') return true;
  if (paper.visibility === 'restricted') return !!user;
  // private:
  if (user && user.id === paper.user_id) return true; // owner
  if (user && await hasGrant(user.id, paper.id)) return true; // granted
  if (shareToken && await validShareLink(paper.id, shareToken))
    return true; // share link
  return false;
}
```

Search combines the FULLTEXT index with optional filter columns, building the query dynamically from whichever filters the user has supplied:

```
// paperController.js (list/search)
let sql = `SELECT * FROM research_papers
  WHERE status='approved' AND is_deleted=FALSE`;
const args = [];
if (q) { sql += ' AND MATCH(title,abstract,journal,sub_category)' +
  ' AGAINST (? IN NATURAL LANGUAGE MODE)'; args.push(q); }
if (category) { sql += ' AND category_id=?'; args.push(category); }
if (year) { sql += ' AND year=?'; args.push(year); }
if (license) { sql += ' AND license=?'; args.push(license); }
sql += ' ORDER BY created_at DESC LIMIT ? OFFSET ?';
args.push(limit, offset);
const [rows] = await pool.query(sql, args);
```

Finally, share-link generation produces a random, unguessable token and stores it with a seven-day expiry, returning a URL the owner can hand to anyone who needs temporary read access:

```
// paperController.js (share link)
const token = crypto.randomBytes(24).toString('hex');
const expires = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000);
await pool.query(
  'INSERT INTO private_share_links (paper_id, token, expires_at) VALUES (?, ?, ?)',
  [paperId, token, expires]);
res.json({ url: `${process.env.CLIENT_URL}/papers/${paperId}?share=${token}` });
```

7. FEATURES

This chapter catalogues the main features of LUMINA and explains how each one works, both from the perspective of the user and in terms of the underlying technical mechanism.

7.1 List of Main Features

16. Seven-step guided Upload Wizard with drag-and-drop file selection.
17. Automatic PDF metadata extraction (title, authors, abstract).
18. SHA-256 duplicate detection on every uploaded file.
19. Administrator moderation workflow (approve / reject / delete) with reasons.
20. Two-factor administrator login via Telegram OTP.
21. Full-text search with category, year, license, and visibility filters.
22. Three-level access control: public, restricted, private.
23. Private, time-limited (7-day) share links.
24. Explicit access-request and grant workflow for private papers.
25. Contributor badge system (newcomer to diamond).
26. Fair weekly view tracking and a trending algorithm.
27. Email and Telegram notifications for key events.
28. Soft deletes and a complete admin audit log.

7.2 How Each Feature Works

Two-factor administrator login

Because an administrator can publish or delete any paper, the admin login is deliberately stronger than the ordinary user login. From the administrator's perspective, login is a two-step experience: they first submit their username and password, and then they are prompted for a six-digit code that arrives on their Telegram within a second or two. Only after the code is entered correctly do they reach the dashboard. Technically, the first step verifies the password and, on success, generates a random OTP stored against the admin record with a five-minute expiry; the second step compares the submitted code and its freshness before issuing the admin JWT. A leaked password is therefore useless to an attacker who does not also control the administrator's Telegram account.

Contributor badges

The badge system gives researchers visible, earned recognition for sustained contribution. From the user's perspective, a small coloured badge appears next to their name and on their profile, advancing as their body of approved work grows. Technically, each successful approval recalculates the author's `approved_count` and maps it to a level: newcomer at zero, bronze at one

or more, silver at five, gold at fifteen, and diamond at thirty. Because the calculation is driven purely by approved papers, the badge cannot be inflated by submitting work that never passes review.

Weekly view tracking and trending

Trending surfaces the papers that are genuinely capturing readers' attention this week. From the user's perspective, the home page shows a trending strip that refreshes as interest shifts. Technically, each view inserts a row into `paper_views` keyed by user, paper, and ISO week, with a UNIQUE constraint that quietly ignores repeat views within the same week. The trending query then ranks papers by their count of distinct weekly views, so a paper rises because many different people are reading it, not because one person refreshed the page repeatedly.

Access requests and share links

For private work, LUMINA offers two complementary sharing mechanisms. A researcher who finds a private paper can send an access request; the owner receives a notification and may grant or deny it, and a granted request is recorded so the requester keeps access thereafter. Alternatively, the owner can proactively generate a share link – a random, unguessable token that grants read access to anyone who holds it for exactly seven days, after which it expires automatically. Together these cover both the “someone asks me” and the “I want to send this to someone” scenarios without ever making the paper public.

Upload Wizard

From the user's perspective, uploading a paper is a calm, seven-step process rather than one intimidating form. Step one accepts a PDF by drag-and-drop; step two extracts and previews metadata; steps three to six let the user confirm the title and authors, add the abstract, keywords, DOI, journal and year, and choose the category, visibility, and license; step six optionally attaches a source-code ZIP; and step seven shows a review screen before submission. Technically, the PDF is first uploaded to a staging folder where it is hashed and checked for duplicates, metadata is extracted with `pdf-parse`, and only on final submission is the file moved into permanent dated storage and a database record created with status 'pending'.

Duplicate detection

Whenever a file reaches the staging area, the server reads its bytes and computes a SHA-256 digest. It then queries `research_papers` for any existing record with the same `file_hash`. If a match is found, the user is warned that an identical document already exists. Because the hash is computed from the file's content, even a renamed copy is caught, which protects the repository against accidental and deliberate duplication.

```
const hash = crypto.createHash('sha256').update(buffer).digest('hex');
const [rows] = await pool.query(
  'SELECT id FROM research_papers WHERE file_hash = ?', [hash]);
```

```
const isDuplicate = rows.length > 0;
```

Moderation workflow and badges

Every submission enters a pending queue. An administrator opens the Paper Manager, reviews the document, and either approves or rejects it (with a reason) or deletes it. On approval the paper's status becomes 'approved', the moderation is written to admin_logs, and a notification (and optional email) is sent to the author. Approval also triggers a recalculation of the author's badge: the system counts their approved papers and assigns newcomer (0), bronze (1+), silver (5+), gold (15+), or diamond (30+). This gives contributors visible, earned recognition.

```
function badgeFor(count) {  
  if (count >= 30) return 'diamond';  
  if (count >= 15) return 'gold';  
  if (count >= 5)  return 'silver';  
  if (count >= 1)  return 'bronze';  
  return 'newcomer';  
}
```

Search, filtering and trending

Readers find papers through a search bar backed by the FULLTEXT index on title, abstract, journal, and sub-category, combined with dropdown filters for category, year, license, and visibility. Results are paginated. Trending papers are ranked by genuine reader interest: each time a paper is viewed, a row is inserted into paper_views keyed by user, paper, and ISO week, with a UNIQUE constraint that prevents a single user from inflating the count by refreshing. The trending list simply ranks papers by their distinct weekly views.

Access control and sharing

Each paper carries a visibility level. Public papers are readable by anyone; restricted papers require a logged-in account; private papers are visible only to the owner, to users who have been explicitly granted access, or to anyone holding a valid share link. A researcher can request access to a private paper, which notifies the owner, who may grant or deny it. Alternatively, an owner can generate a random share token that grants read access for seven days before automatically expiring.

Notifications

LUMINA keeps users informed through a unified notification system. Whenever a meaningful event occurs – a paper is approved, rejected, or deleted, or someone requests or is granted access – a row is written to the notifications table and surfaced on the user's dashboard with an unread indicator. Where SMTP and Telegram are configured, the same event is also pushed by email and as a Telegram message, so contributors learn the outcome of their submission promptly without having to keep checking the site.

Administrator analytics

The admin dashboard presents a live picture of the repository's health: total papers, pending submissions, registered users, and total views, alongside charts of submissions over time and a breakdown of papers by category rendered with Recharts. Recent moderation activity is listed from the audit log, giving administrators an at-a-glance sense of what has happened and what needs attention.

Feature summary

The following table maps each headline feature to the primary technical mechanism that delivers it.

Feature	Underlying mechanism
Upload Wizard	Multi-step React form + staged Multer uploads
Metadata extraction	pdf-parse over the staged PDF
Duplicate detection	SHA-256 hash compared against file_hash
Moderation	Status field + admin_logs audit trail
Admin 2FA	Telegram OTP with 5-minute expiry
Search	MySQL FULLTEXT index + filter columns
Access control	Visibility enum + grants + share tokens
Badges	approved_count thresholds
Trending	Distinct weekly views in paper_views
Notifications	notifications table + email + Telegram

8. TESTING

Testing was carried out throughout development to confirm that each part of LUMINA behaves correctly in isolation and that the parts work together end-to-end. The strategy combined manual API testing, integration testing of complete flows, and validation against a set of representative test cases.

8.1 Postman Testing (Frontend / Backend)

Postman was the primary tool for exercising the backend API independently of the frontend. Each endpoint was tested directly by constructing requests with the appropriate method, headers, and body. Authentication tokens obtained from the login endpoints were stored in Postman environment variables and reused across protected requests, which made it easy to verify that authorization rules were enforced. File-upload endpoints were tested using Postman's form-data mode to attach real PDF and ZIP files.

For each endpoint the team verified three things: that valid input produced the expected success response and status code; that invalid input (missing fields, wrong types, malformed tokens) produced a clear error with the correct status; and that protected endpoints rejected unauthenticated or under-privileged requests. For example, calling an admin route with a researcher's token correctly returned 403 Forbidden.

8.2 Integration Testing

Integration testing verified complete user journeys that span the frontend, backend, database, and file system. The most important flow tested was the full paper lifecycle: a researcher registers, logs in, uploads a PDF through the Upload Wizard, sees metadata extracted, submits the paper, and the record appears in the pending queue; an administrator then logs in with OTP, approves the paper, and the author receives a notification while the paper becomes visible in the public listing and the author's badge updates. Each of these steps was run against a real database to confirm that data flowed correctly through every layer.

Other integration flows tested included the private-paper access-request cycle (request, owner notification, grant, access), the share-link cycle (generate, access within seven days, expiry after), and the rejection flow (reject with reason, author notification, paper not published).

8.3 Tools Used for Testing

Tool	Purpose
Postman	Manual testing of individual REST endpoints and authorization
Browser DevTools	Inspecting network requests, console errors, and React state
MySQL Workbench	Verifying that database rows were created and updated correctly

Tool	Purpose
Manual UI walkthroughs	End-to-end testing of complete user journeys in the browser

8.4 Test Cases and Results

A representative subset of the test cases executed during development is presented below, together with their outcomes.

TC	Description	Expected Result	Status	Result
TC-01	Register with valid details	Account created, token returned	Done	Pass
TC-02	Register with existing email	Rejected with duplicate error	Done	Pass
TC-03	Login with correct password	Token + user returned	Done	Pass
TC-04	Login with wrong password	401 Invalid credentials	Done	Pass
TC-05	Admin login then valid OTP	Admin token issued	Done	Pass
TC-06	Admin login then expired OTP	Rejected, no token	Done	Pass
TC-07	Upload duplicate PDF	Duplicate warning shown	Done	Pass
TC-08	Extract metadata from PDF	Title/authors pre-filled	Done	Pass
TC-09	Submit paper	Status 'pending' created	Done	Pass
TC-10	Approve paper as admin	Status 'approved', badge updated	Done	Pass
TC-11	Researcher opens admin route	403 Forbidden	Done	Pass
TC-12	Guest views private paper	Access denied	Done	Pass
TC-13	Access via valid share link	Paper readable	Done	Pass
TC-14	Access via expired share link	Access denied	Done	Pass
TC-15	Search by keyword	Relevant papers returned	Done	Pass

In addition to the cases above, a second batch of tests exercised access control, notifications, and data integrity. These confirmed that the system behaves correctly at its security boundaries, which are the most important to get right.

TC	Description	Expected Result	Status	Result
TC-16	Request access to private paper	Owner notified, request pending	Done	Pass
TC-17	Owner grants access request	Requester gains access + notice	Done	Pass
TC-18	Owner denies access request	Access remains blocked	Done	Pass
TC-19	Reject paper with reason	Author notified, not published	Done	Pass
TC-20	Delete paper (soft)	Hidden from listings, log kept	Done	Pass
TC-21	Repeat view in same week	View count not double-counted	Done	Pass
TC-22	Approve 5th paper	Badge upgrades to silver	Done	Pass
TC-23	Filter by category + year	Correct subset returned	Done	Pass
TC-24	Submit without required field	Validation error shown	Done	Pass
TC-25	Token tampered / invalid	401 Unauthorized	Done	Pass

Across all twenty-five documented test cases the system behaved as expected, with particular attention paid to the authorization boundaries where a defect would have the most serious consequences. The combination of endpoint-level testing in Postman and full-journey testing in the browser gave confidence that both the individual pieces and the assembled system are correct.

8.5 Testing Summary and Coverage

The testing effort was organised by module so that no major area of functionality was left unexercised. The table below summarises the coverage achieved across the system.

Module	Cases	Focus of testing
Authentication	TC-01 to TC-06	Registration, login, admin 2FA, error paths
Upload pipeline	TC-07 to TC-09, TC-24	Duplicate detection, extraction, validation
Moderation	TC-10, TC-19, TC-20, TC-22	Approve, reject, delete, badge update
Authorization	TC-11, TC-12, TC-25	Role checks, private access, bad tokens

Module	Cases	Focus of testing
Access & sharing	TC-13, TC-14, TC-16 to TC-18	Share links, requests, grant/deny
Discovery	TC-15, TC-21, TC-23	Search, filters, fair view counting

Every module reached a state in which its core behaviour and its principal error paths were verified. Where a test revealed a defect – for example an early version that returned 500 instead of 403 for an unauthorised request – the underlying code was corrected and the case re-run until it passed. This disciplined loop of test, fix, and re-test is what gives confidence that the delivered system is dependable.

9. DEPLOYMENT

This chapter describes how LUMINA is taken from source code to a running, accessible application, covering the deployment steps, the environment configuration required, and the hosting of the frontend and backend.

9.1 Steps to Deploy

Deployment of LUMINA follows a repeatable sequence:

8. Provision a server with Node.js and a MySQL instance, and create the application database.
9. Clone the repository from GitHub onto the server.
10. Install backend dependencies with `npm install` in the server directory.
11. Create the `.env` file with database credentials, the JWT secret, and the SMTP and Telegram tokens.
12. Run the schema script (`schema.sql`) or let `ensureRuntimeSchema` create the tables on startup, then seed the initial admin account.
13. Start the API behind a process manager such as PM2 so it restarts automatically on failure.
14. Build the frontend with `npm run build` in the client directory, producing static assets in `dist/`.
15. Serve the built frontend through a static host or a reverse proxy, and point the proxy's `/api` path at the Node server.

9.2 Environment Configuration

All environment-specific and secret values are supplied through environment variables, never hard-coded. The backend reads its configuration from a `.env` file via `dotenv`. The key variables are listed below.

```
# server .env
PORT=5000
CLIENT_URL=https://lumina.example.edu
DB_HOST=localhost
DB_USER=lumina_user
DB_PASSWORD=*****
DB_NAME=lumina_db
JWT_SECRET=*****
JWT_EXPIRES_IN=7d
TELEGRAM_BOT_TOKEN=*****
TELEGRAM_CHAT_ID=*****
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=noreply@lumina.app
```

```
SMTP_PASS=*****  
  
# client .env  
VITE_API_URL=https://lumina.example.edu/api
```

Because the JWT secret, database password, and external-service tokens are read from the environment, the same codebase can be deployed safely to development, staging, and production simply by supplying different values – and none of these secrets is ever committed to version control.

9.3 Hosting of Frontend and Backend

The frontend and backend are hosted as separate concerns. The React frontend compiles to static files and can be served by any static-hosting platform or by a CDN, which gives fast global delivery and trivial scaling. The Node.js backend runs as a persistent process on a server or virtual machine, sitting behind an Nginx reverse proxy that terminates HTTPS and forwards API requests to port 5000. The MySQL database runs either on the same machine for small deployments or on a dedicated managed database service for larger ones. Uploaded PDFs and ZIP files are stored on a persistent disk attached to the backend server, organised into the dated folder structure described earlier.

Deployment checklist

Before a deployment is considered complete, the following checklist is verified to ensure the running system is secure and correct.

Check	Status
HTTPS enabled and HTTP redirected	Required
Strong, unique JWT_SECRET set in environment	Required
Database user limited to the application schema only	Required
CORS restricted to the production frontend origin	Required
Initial admin account seeded and Telegram chat id set	Required
Uploads directory on a persistent, backed-up volume	Required
Process manager configured to restart on failure	Recommended
Database backups scheduled	Recommended

10. CHALLENGES & LIMITATIONS

No real project is free of obstacles. This chapter honestly records the main difficulties encountered while building LUMINA, the solutions adopted, and the limitations that remain in the current version.

10.1 Issues Faced During Development

Reliable PDF metadata extraction: PDFs are notoriously inconsistent. Some embed clean metadata while others are scanned images or store the title as ordinary body text. Early versions of the extractor frequently returned empty or wrong titles.

Securing the admin login: A simple username-and-password admin login felt too weak for an account that can delete any paper. Designing a second factor that was secure but did not require paid SMS infrastructure was a real challenge.

Preventing view-count gaming: An early trending implementation simply incremented a counter on each page load, which meant a single user could push their own paper up the rankings just by refreshing.

Managing complex multi-step form state: The seven-step Upload Wizard had to preserve data across steps, allow the user to go back and edit, and coordinate two separate file uploads, which made its state management intricate.

Enforcing access control consistently: With three visibility levels, owner access, granted access, and share links, it was easy to leave a gap through which private content could leak.

Coordinating two developers on a shared codebase: Working in parallel on the frontend and backend occasionally produced merge conflicts and mismatched assumptions about the shape of API responses, which could break the integration between the two halves of the system.

10.2 Solutions Applied

Metadata extraction: The pdf-parse library was used to pull text, and the wizard always lets the user review and correct the extracted title, authors, and abstract before submission, so an imperfect extraction never blocks the upload.

Admin 2FA: The Telegram Bot API was adopted to deliver six-digit one-time passwords for free. Each OTP is single-use and expires after five minutes, giving strong protection without paid services.

Fair view tracking: Views are recorded in a paper_views table keyed by user, paper, and ISO week, with a UNIQUE constraint that silently ignores repeat views in the same week, so trending reflects distinct readers.

Wizard state: A single state object in the parent wizard component holds all fields and the current step, with controlled inputs at each step, making back-and-forth navigation and review reliable.

Centralised access checks: All access decisions were funnelled into one place in the paper-detail controller, which evaluates ownership, grants, and share tokens together, removing inconsistent ad-hoc checks.

Disciplined Git workflow and a shared API contract: The team adopted short-lived feature branches, frequent pulls from the main branch, and an agreed, documented shape for every API response. Defining the contract up front meant the frontend could be built against a known interface and merges stayed small and predictable.

10.3 Current Limitations or Known Bugs

- Plagiarism detection is limited to exact-duplicate file hashing; it cannot detect paraphrased or partially copied content.
- Metadata extraction works only on text-based PDFs; scanned image PDFs are not processed by OCR.
- Uploaded files are stored on the local server disk rather than on dedicated cloud object storage, which limits horizontal scaling.
- Notifications are delivered in-app, by email, and by Telegram, but there is no real-time push to the browser; the user sees new notifications on their next page load.
- The search uses MySQL FULLTEXT, which is effective but less sophisticated than a dedicated search engine for ranking and typo tolerance.

These limitations are summarised below alongside the planned direction for addressing each. None of them undermines the core functionality of the system; rather, they mark the natural boundary between what was achievable within the project's scope and the enhancements that would follow in a production rollout.

Current limitation	Planned direction
Only exact-duplicate detection	Text-similarity plagiarism scoring
No OCR for scanned PDFs	Add an OCR pass before extraction
Local disk file storage	Move to cloud object storage
Notifications on next load	Real-time push via WebSockets
FULLTEXT search ranking	Adopt a dedicated search engine

11. FUTURE ENHANCEMENTS

LUMINA was built with extension in mind. Its modular structure means new capabilities can be added without disturbing the existing system. This chapter outlines the planned features and the integrations and optimisations that would most improve the platform.

11.1 Planned Features

Advanced plagiarism detection: Move beyond exact-hash matching to text-similarity analysis that can flag paraphrased or partially copied submissions, giving administrators a similarity score during review.

Citation graph and analytics: Allow papers to cite other papers in the repository, then visualise the resulting citation network and compute simple impact metrics for each contributor.

Recommendation engine: Suggest related papers to a reader based on the categories, keywords, and authors of what they are currently viewing, increasing discovery of relevant work.

OCR for scanned PDFs: Add optical character recognition so that scanned or image-based papers can also have their metadata extracted and their text indexed for search.

Real-time notifications: Use WebSockets or server-sent events to push approval, rejection, and access-request notifications to the user instantly rather than on the next page load.

Public researcher profiles: Give each researcher a shareable public profile page listing their approved papers, badge level, and linked scholar identifiers.

11.2 Possible Integrations or Optimisations

- Cloud object storage (such as Amazon S3) for uploaded files, enabling the backend to scale horizontally and improving durability.
- A dedicated search engine (such as Elasticsearch or MeiliSearch) for faster, typo-tolerant, relevance-ranked search.
- ORCID and Google Scholar integration so that authors can link verified external identities and import their publication lists.
- DOI minting so that papers accepted into the repository can be assigned a permanent, citable identifier.
- Containerisation with Docker and a CI/CD pipeline for automated testing and one-command deployment.
- Caching of frequently accessed listings and trending results to reduce database load under heavy traffic.

If these enhancements were to be scheduled, they would naturally be prioritised by the value they add against the effort they require. The indicative prioritisation below places the highest-value, most-contained improvements first.

Enhancement	Value	Effort
Real-time notifications	High	Low
Public researcher profiles	High	Low
Cloud object storage	High	Medium
Dedicated search engine	High	Medium
Recommendation engine	Medium	Medium
Similarity-based plagiarism	High	High
Citation graph & analytics	Medium	High
OCR for scanned PDFs	Medium	High

This roadmap shows that LUMINA is not a finished, closed product but a living foundation. Its modular architecture means that each of these enhancements can be added as an independent unit of work without disturbing the parts already in place, which is exactly the property a long-lived institutional system needs.

12. CONCLUSION

12.1 Summary of the Project

LUMINA is a complete, full-stack Academic Research Repository that brings the scattered research output of an institution into a single, secure, searchable, and moderated platform. Built with React and Vite on the frontend and Node.js, Express, and MySQL on the backend, it covers the entire lifecycle of a research paper: a guided multi-step upload with automatic metadata extraction and duplicate detection, a rigorous administrator moderation workflow protected by two-factor authentication, fine-grained access control across public, restricted, and private visibility levels, and a discovery experience powered by full-text search, filters, and a fair trending algorithm. A contributor badge system and a comprehensive audit log round out the platform.

12.2 What Was Achieved

The project successfully delivered every objective set out at the start. A working, end-to-end system was implemented in which researchers can register, submit, and manage papers, and administrators can review and publish them. Security goals were met through bcrypt password hashing, JWT sessions, parameterised queries, SHA-256 file fingerprinting, and Telegram-based two-factor admin login. The access-control model was implemented consistently across the API, and the contributor recognition and trending mechanisms were built to be fair and resistant to gaming. The result is a reliable, responsive web application that demonstrably solves the problems identified in the introduction.

- A fully functional repository with a nine-table relational schema and a complete REST API.
- A polished, responsive React interface with role-based navigation and protected routes.
- A robust security model spanning authentication, authorization, and data integrity.
- A moderation and audit system that keeps the published collection trustworthy.

12.3 Skills Learned During Development

Building LUMINA was a substantial learning experience that strengthened the team's abilities across the full stack.

- Full-stack web development: designing and connecting a React frontend, an Express API, and a MySQL database into one coherent system.
- RESTful API design: structuring clear, resource-oriented endpoints with appropriate methods, status codes, and middleware.
- Relational database design: modelling entities and relationships, choosing keys and indexes, and writing safe parameterised queries.

- Web security: implementing password hashing, JWT sessions, role-based authorization, and two-factor authentication.
- React state management: applying Context, custom hooks, controlled components, and lifting state up in a real application.
- Collaboration and version control: working as a team through Git and GitHub with branches and code review.

Project timeline

The work progressed through clearly defined phases over the course of the project, each building on the last.

Phase	Work completed
Requirements & design	Problem analysis, scope definition, database schema and ER design
Authentication	User registration/login, JWT sessions, admin 2FA over Telegram
Upload pipeline	Multi-step wizard, staged uploads, hashing, metadata extraction
Moderation console	Pending queue, approve/reject/delete, audit logging, badges
Discovery	Browse, full-text search, filters, trending, paper detail
Access control	Public/restricted/private, access requests, share links
Testing & polish	Postman and integration testing, UI refinement, documentation

Team contributions

The project was a genuine collaboration, with responsibilities shared across the full stack so that both members gained end-to-end experience.

Member	Primary contributions
Nimma Lokesh Reddy	Backend API, database schema, authentication and 2FA, moderation and audit logging, deployment configuration
Ramagiri Rishik Rao	React frontend, Upload Wizard, search and discovery UI, admin dashboard and charts, integration testing

In closing, LUMINA stands as a complete and working demonstration of how a thoughtfully designed full-stack application can solve a real institutional problem. It not only meets its stated objectives but also lays a clean foundation on which the future enhancements described in the previous chapter can be built.

13. REFERENCES

The following books, official documentation sites, tutorials, and APIs were referred to during the design and development of LUMINA.

- [1] React Official Documentation – <https://react.dev>
- [2] Vite Official Documentation – <https://vitejs.dev>
- [3] React Router v6 Documentation – <https://reactrouter.com>
- [4] Node.js Official Documentation – <https://nodejs.org/en/docs>
- [5] Express.js Official Documentation – <https://expressjs.com>
- [6] MySQL 8.0 Reference Manual – <https://dev.mysql.com/doc>
- [7] mysql2 Driver Documentation – <https://github.com/sidorares/node-mysql2>
- [8] JSON Web Token (JWT) Introduction – <https://jwt.io/introduction>
- [9] bcrypt.js Library – <https://github.com/dcodeIO/bcrypt.js>
- [10] Multer File Upload Middleware – <https://github.com/expressjs/multer>
- [11] pdf-parse Library – <https://www.npmjs.com/package/pdf-parse>
- [12] Axios HTTP Client Documentation – <https://axios-http.com>
- [13] Nodemailer Documentation – <https://nodemailer.com>
- [14] Telegram Bot API – <https://core.telegram.org/bots/api>
- [15] Recharts Documentation – <https://recharts.org>
- [16] MDN Web Docs (HTML, CSS, JavaScript) – <https://developer.mozilla.org>
- [17] arXiv API Documentation – <https://info.arxiv.org/help/api>
- [18] OWASP Web Security Guidelines – <https://owasp.org>

14. APPENDICES

This section is reserved for supporting material that will be added during the practical demonstration and final compilation of the report. The headings below mark where each item will be placed; the content for each will be inserted in the practical sessions.

Appendix A – Screenshots of the Application

(Screenshots of the Home page, Browse / All Papers page, Upload Wizard steps, Paper Detail view, Researcher Dashboard, and the Admin Dashboard will be placed here.)

Appendix B – Sample Code Snippets

(Representative source-code listings from the frontend and backend will be placed here.)

Appendix C – Installation / Setup Instructions

(Step-by-step instructions to install dependencies, configure the environment, and run the project locally will be placed here.)

Appendix D – User Manual / Guide

(A short user guide explaining how researchers and administrators use the platform will be placed here.)

Appendix E – Geo Tag Photos with Guide

(Geo-tagged photographs taken with the project guide will be placed here.)

Appendix F – Review Forms with Guide Signatures

(Scanned project review forms bearing the guide's signatures will be placed here.)